

WAWUBASKET: WORKFLOW

EXECUTIVE SUMMARY

Parameter	Value
Timeline	30 calendar days
Total Functions	129 across 6 user types
API Endpoints	74
Database Tables	18
Third-party Integrations	8 (training, banking, logistics, government)
Security Controls	22
Code Generation	80% AI-assisted, 20% human integration

PART 2: COMPLETE FUNCTIONALITY MASTER LIST (129 FUNCTIONS)

User Functions: Mobile + Web (34 Functions)

#	FUNCTION	CATEGORY	DEVICE	API ENDPOINT
1	Phone number registration	Onboarding	Both	POST /auth/register
2	Email registration (fallback)	Onboarding	Both	POST /auth/register
3	OTP send/verify	Onboarding	Both	POST /auth/otp/send, /auth/otp/verify
4	Biometric login (Face ID/Fingerprint)	Onboarding	Mobile	POST /auth/biometric
5	Role selection (user/vendor/rider/agent)	Onboarding	Both	PUT /users/role
6	Referral code entry	Onboarding	Both	POST /referrals/apply
7	4 vertical pills (Restaurants/Produce/Livestock/Essentials)	Browsing	Both	GET /vendors?vertical=
8	Global search (products + vendors)	Browsing	Both	GET /search?q=
9	Vendor list with sort/filter	Browsing	Both	GET /vendors?sort=&filter=
10	Map view of vendors	Browsing	Both	GET /vendors/nearby?lat=&lng=
11	Product detail with modifiers	Browsing	Both	GET /products/{id}
12	Special instructions (max 200 chars)	Browsing	Both	POST /cart/items/{id}/instructions

13	Quantity selector (stepper 1-20)	Cart	Both	PUT /cart/items/{id}/quantity
14	Vendor-separated cart	Cart	Both	GET /cart
15	Delivery slot picker	Checkout	Both	GET /delivery-slots
16	Saved addresses (CRUD)	Checkout	Both	GET/POST/PUT/DELETE /addresses
17	Multiple payment methods	Checkout	Both	POST /payments/initialize
18	Buy Now Pay Later (BNPL)	Checkout	Both	POST /loans/apply
19	Order tracking (live map + timeline)	Orders	Both	WS /tracking/{orderId}
20	Push notifications for status changes	Orders	Mobile	POST /notifications/register
21	Order history with pagination	Orders	Both	GET /orders? Page=&limit=
22	One-tap reorder	Orders	Both	POST /orders/{id}/reorder
23	Cancel order (before "prepared")	Orders	Both	POST /orders/{id}/cancel
24	Report issue with photo	Orders	Both	POST /disputes
25	Farmer listing (post inventory for export)	Cross-border	Both	POST /exports/listings
26	Real-time prices across corridors	Cross-border	Both	GET /exports/prices? Corridor=
27	Escrow deposit (CFA/Naira)	Cross-border	Both	POST /escrow/hold
28	Confirm delivery (SMS/App)	Cross-border	Both	POST /escrow/release
29	View credit score (0-1000)	Cross-border	Both	GET /credit-score
30	Request transport (post load)	Cross-border	Both	POST /transport/loads
31	Language selector (5 languages)	Profile	Both	PUT /users/language
32	Delete account (GDPR)	Profile	Both	DELETE /users/account
33	WAWU+ subscription	Profile	Both	POST /subscriptions
34	Notification preferences	Profile	Both	PUT /users/notifications

VENDOR FUNCTIONS: WEB DASHBOARD (36 FUNCTIONS)

#	FUNCTION	CATEGORY	API ENDPOINT
35	Business registration (multi-step)	Onboarding	POST /vendors/register
36	Document upload	Onboarding	POST /vendors/documents
37	Location pin placement	Onboarding	PUT /vendors/location
38	Bank account verification	Onboarding	POST /vendors/bank-verify
39	Open/close toggle	Daily Ops	PUT /vendors/status
40	Live order dashboard	Daily Ops	GET /vendors/orders/live
41	Accept/decline order	Daily Ops	PUT /vendors/orders/{id}/accept

42	Mark order ready	Daily Ops	PUT /vendors/orders/{id}/ready
43	Print order ticket	Daily Ops	GET /vendors/orders/{id}/ticket
44	Sales today (metric cards)	Daily Ops	GET /vendors/dashboard/today
45	Add item	Catalog	POST /vendors/products
46	Bulk upload items (CSV)	Catalog	POST /vendors/products/bulk
47	Out of stock toggle	Catalog	PUT /vendors/products/{id}/stock
48	Item cloning	Catalog	POST /vendors/products/{id}/clone
49	Category management	Catalog	CRUD /vendors/categories
50	Stock level input	Inventory	PUT /vendors/inventory/{id}
51	Low stock alert	Inventory	POST /vendors/alerts/low-stock
52	Batch tracking (harvest/slaughter date)	Inventory	PUT /vendors/batches/{id}
53	Price per unit management	Inventory	PUT /vendors/products/{id}/price
54	Daily sales summary	Financials	GET /vendors/analytics/sales
55	Payout history	Financials	GET /vendors/payouts
56	Request payout	Financials	POST /vendors/payouts/request
57	Commission breakdown	Financials	GET /vendors/commissions
58	Invoice download (PDF)	Financials	GET /vendors/invoices/{month}
59	Rating & reviews	Analytics	GET /vendors/reviews
60	Order volume trend	Analytics	GET /vendors/analytics/trend
61	Peak hours heatmap	Analytics	GET /vendors/analytics/heatmap
62	Customer geography map	Analytics	GET /vendors/analytics/geography
63	Cancel rate with reasons	Analytics	GET /vendors/analytics/cancellations
64	Store hours	Settings	PUT /vendors/hours
65	Preparation time	Settings	PUT /vendors/prep-time
66	Delivery radius	Settings	PUT /vendors/radius
67	Holiday mode	Settings	POST /vendors/holidays
68	Staff accounts (sub-users)	Settings	CRUD /vendors/staff
69	Notification settings	Settings	PUT /vendors/notifications
70	Menu export (backup)	Settings	GET /vendors/menu/export

AGENT FUNCTIONS: MOBILE OFFLINE (16 FUNCTIONS)

#	FUNCTION	CATEGORY	API ENDPOINT
71	Agent registration (device fingerprinting)	Onboarding	POST /agents/register
72	Register trader offline	Trader Mgmt	POST /agents/traders/offline
73	Capture trader photo + ID	Trader Mgmt	POST /agents/traders/photo
74	Sync data (conflict resolution)	Trader Mgmt	POST /agents/sync
75	Enter sale details	Transactions	POST /agents/transactions/offline
76	Capture photo of goods	Transactions	POST /agents/transactions/photo
77	Offline storage (10,000 txns)	Transactions	Local SQLite
78	Record cash payout	Cash	POST /agents/cash-payout
79	Capture trader signature/thumbprint	Cash	POST /agents/signature
80	WAWU reimburses agent	Cash	POST /agents/reimburse
81	View commission earned	Earnings	GET /agents/commission
82	Daily/weekly earnings dashboard	Earnings	GET /agents/earnings
83	Receive push notifications	Communication	POST /agents/notifications
84	WhatsApp group integration	Communication	External (Meta API)
85	View pending sync count	Offline	GET /agents/sync/status
86	Trader search (by phone/name)	Trader Mgmt	GET /agents/traders/search

TRANSPORT FUNCTIONS: DRIVER APP (12 FUNCTIONS)

#	Function	Category	API Endpoint
87	Register transport union	Union Mgmt	POST /transport/unions
88	Define covered routes	Union Mgmt	POST /transport/routes
89	List available trucks	Union Mgmt	GET /transport/trucks
90	View available loads	Loads	GET /transport/loads/available
91	Bid on load	Loads	POST /transport/loads/{id}/bid
92	Accept assigned load	Loads	PUT /transport/loads/{id}/accept
93	Log trip start (GPS)	Trip	POST /transport/trips/{id}/start
94	Log checkpoint arrival	Trip	POST /transport/trips/{id}/checkpoint
95	Log delivery completion	Trip	POST /transport/trips/{id}/complete
96	Receive digital payment	Payment	GET /transport/earnings
97	View trip earnings	Payment	GET /transport/earnings/trips
98	Contact trader (masked phone)	Communication	POST /transport/contact/{traderId}

GOVERNMENT FUNCTIONS: DATA PORTAL (9 FUNCTIONS)

#	Function	Category	API Endpoint
99	Anonymized aggregate trade data	Data API	GET /data/trade/aggregate
100	Real-time border dashboard	Dashboard	GET /data/border/live
101	Monthly PDF report download	Reports	GET /data/reports/monthly
102	CSV export (raw anonymized)	Reports	GET /data/export/csv
103	STR digital backbone integration	Government	POST /data/str/sync
104	Policy simulation tool	Policy	POST /data/policy/simulate
105	Multi-user government accounts (RBAC)	Auth	POST /data/accounts
106	API keys with expiry + IP whitelist	Security	POST /data/api-keys
107	Audit log of data pulls	Audit	GET /data/audit

ADMIN FUNCTIONS: SUPER ADMIN WEB (22 FUNCTIONS)

#	Function	Category	API Endpoint
108	Approve vendors	User Mgmt	PUT /admin/vendors/{id}/approve
109	Approve agents	User Mgmt	PUT /admin/agents/{id}/approve
110	Approve riders	User Mgmt	PUT /admin/riders/{id}/approve
111	Suspend/ban users	User Mgmt	PUT /admin/users/{id}/suspend
112	Commission override	System	PUT /admin/vendors/{id}/commission
113	Zone management (geo-fences)	System	POST /admin/zones
114	System-wide promo creation	System	POST /admin/promos
115	Feature toggles	System	PUT /admin/features
116	Ticket system (disputes)	Disputes	GET /admin/disputes
117	Review evidence	Disputes	GET /admin/disputes/{id}/evidence
118	Final resolution (release/refund)	Disputes	POST /admin/disputes/{id}/resolve
119	Manual escrow adjustment	Financial	POST /admin/escrow/adjust
120	Payout hold/unhold	Financial	PUT /admin/payouts/{id}/hold
121	Reconciliation dashboard	Financial	GET /admin/reconciliation
122	Audit log viewer	Audit	GET /admin/audit
123	Export audit log	Audit	GET /admin/audit/export
124	Admin MFA (mandatory)	Security	POST /admin/mfa/setup
125	System health dashboard	Monitoring	GET /admin/health
126	Real-time transaction volume	Monitoring	WS /admin/metrics/live
127	Alert configuration	Monitoring	POST /admin/alerts
128	Backup management	Ops	POST /admin/backup
129	User impersonation (support mode)	Support	POST /admin/impersonate/{userId}

WORKFLOW

ROLE	TASKS	COMMANDS / CODE GENERATION PROMPT	DELIVERABLE
Backend	Setup Node.js + Express + PostgreSQL + Redis. Generate database schema (18 tables) using Prisma.	npx prisma init → paste schema from Part 3 → npx prisma migrate dev	Working local database with migrations
Frontend Web	Setup React + Vite + TailwindCSS + Zustand. Configure routing.	npm create vite@latest wawu-web -- --template react-ts → npm install zustand react-router-dom tailwindcss	Blank web app running on localhost:5173
Mobile	Setup React Native Expo + React Navigation.	npx create-expo-app wawu-mobile → npm install @react-navigation/native	Blank mobile app on emulator
DevOps	Setup GitHub repo with branch protection. Configure Docker Compose.	git init → Create Dockerfile + docker-compose.yml with PostgreSQL, Redis, backend service	Repo + Docker running

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate User Service: register, login, OTP send/verify, JWT + refresh token.	"Generate Node.js Express API for user authentication: phone/email registration, OTP via SMS simulator, JWT access token (15min expiry) and refresh token (7 days), bcrypt password hashing, Redis blacklist for logout"	Auth APIs (POST /auth/register, /auth/login, /auth/otp/send, /auth/otp/verify, /auth/refresh, /auth/logout)
Frontend Web	Generate login/signup/OTP screens. Axios interceptor for JWT.	"Generate React component for phone login with OTP verification, Zustand store for auth state, Axios interceptor to attach Bearer token"	Auth pages working
Mobile	Generate same screens + biometric login using Expo LocalAuthentication.	"Generate React Native Expo screen for login with biometric (Face ID/Fingerprint) fallback to OTP, AsyncStorage for token persistence"	Auth + biometric
DevOps	Configure ESLint + Prettier + Husky pre-commit hooks.	npx eslint --init → npm install -D prettier husky lint-staged → Configure .husky/pre-commit	Code quality enforced

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate RBAC: 6 roles with middleware for endpoint protection.	"Generate Express middleware for role-based access control: roles = ['user','vendor','agent','rider','admin','government']. Create decorator @RequireRole(role) for routes. Add user-ownership check for resources."	Role-based endpoint protection
Frontend Web	Build role-based dashboard routing	"Generate React Router configuration that conditionally renders routes based on user role from Zustand store: vendor → /vendor/, admin → /admin/, government → /government/*"	Conditional rendering
Mobile	Build role-based navigation (agent sees agent app).	"Generate React Navigation conditional screens: if role === 'agent', show agent stack; if role === 'rider', show rider stack; else show user stack"	Conditional navigation
DevOps	Setup development AWS environment (free tier). SSL via Let's Encrypt.	aws configure → terraform apply for t2.micro EC2 → certbot --nginx -d dev.wawu.africa	Dev environment URL

End of Phase 1: Authentication working. Users can register, login, access role-appropriate dashboards.

Phase 2: Core User Flows

Browsing & Discovery

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Vendor Service (CRUD) + Search Service (Elasticsearch).	"Generate Express API for vendors: CRUD operations, GET /vendors with filters (vertical, rating, price range), GET /vendors/nearby using PostGIS. Generate search service using Elasticsearch for products and vendors."	Vendor + search APIs
Frontend Web	Build vendor listing page with 4 vertical pills.	"Generate React component VendorList with 4 category pills (Restaurants/Produce/Livestock/Essentials), sort dropdown (rating/distance/price), filter sidebar, responsive grid layout"	Listing page
Mobile	Same as web + bottom navigation bar.	"Generate React Native screen with bottom tab navigator (Home, Search, Cart, Orders, Profile), FlatList for vendors, category pills horizontal scroll"	Listing with bottom nav
DevOps	Setup staging environment.	kubectl create namespace staging → Deploy backend + frontend → Configure staging.wawu.africa DNS	Staging URL

Product Details & Cart Foundation

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Product Service + Cart Service.	"Generate APIs: products CRUD with variants/modifiers, cart: POST /cart (add item), PUT /cart/items/:id (update quantity), DELETE /cart/items/:id, GET /cart, POST /cart/merge (guest → user)"	Product + cart APIs
Frontend Web	Build product detail page.	"Generate React component ProductDetail: image carousel, product info, modifier options (radio/checkbox), quantity stepper (1-20), add to cart button with animation, special instructions textarea (200 char limit)"	Product page
Mobile	Same + add to cart animation.	"Generate React Native ProductDetail screen with Image slider, modifier buttons, quantity selector, 'Add to Cart' with spring animation and cart badge update"	Product + animation
DevOps	Setup logging (Winston) + error tracking (Sentry).	npm install winston @sentry/node → Configure log rotation (7 days) → npm install @sentry/react for frontend	Error monitoring

Cart Management

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate cart merge logic (guest → user).	"Generate cart merge function: when user logs in, fetch guest cart from localStorage (sent in request header), merge items (same product + modifiers = combine quantity), delete guest cart"	Cart merge API
Frontend Web	Build cart page with vendor-separated groups.	"Generate React component CartPage: vendor-separated sections, each with items (image, name, quantity, price), edit quantity inline, remove item button, clear cart button, subtotal per vendor"	Cart page
Mobile	Same + swipe to delete.	"Generate React Native CartScreen: SectionList grouped by vendor, each item with Swipeable row to delete, quantity stepper, checkout button"	Cart with swipe
DevOps	Setup Redis for session + cart caching.	docker run -d --name redis -p 6379:6379 redis:alpine → Configure express-session with Redis store	Redis configured

Checkout Flow

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Order Service (state machine) + Address API.	"Generate order state machine: pending → confirmed → preparing → ready → picked_up → en_route → delivered. Address CRUD API with geocoding (Mapbox). GET /delivery-slots (30-min windows from 8am-10pm)."	Order + address APIs
Frontend Web	Build checkout flow.	"Generate React multi-step checkout: Step1 Address (saved addresses + add new with Mapbox autocomplete), Step2 Delivery Slot (calendar + time slots), Step3 Order Summary (items, fees, total)"	Checkout screens
Mobile	Same + contacts import for addresses.	"Generate React Native checkout screens + expo-contacts to import address from device contacts"	Checkout + contacts
DevOps	Implement rate limiting (100 req/min per IP).	"Configure express-rate-limit with Redis store: 100 requests per minute per IP, 429 response with Retry-After header"	Rate limiting active

Payment Integration

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Integrate Paystack/Flutterwave. Generate escrow logic.	"Integrate Paystack: POST /payments/initialize (create transaction reference), POST /payments/verify (webhook handler with signature verification). Escrow: POST /escrow/hold (hold funds), POST /escrow/release (release after delivery confirmation). BNPL: POST /loans/apply (send to microfinance partner via webhook)"	Payment + escrow APIs
Frontend Web	Integrate payment widget.	"Generate React payment modal: show payment method selector (card, bank transfer, USSD, mobile money), initialize Paystack popup, handle success/failure callbacks"	Payment working
Mobile	Same + Apple Pay/Google Pay.	"Generate React Native payment screen with Paystack SDK, Apple Pay (iOS) and Google Pay (Android) integrations"	Mobile payment
DevOps	PCI DSS compliance: tokenize cards, never log PAN.	Verify Paystack tokenization → Ensure no card details in logs → Add log scrubber for PCI data → Run PCI self-assessment questionnaire	Payment Partner

Order Tracking (SHIPBUBBLE)

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Tracking Service (WebSocket) + Push Notifications.	"Generate WebSocket server using Socket.io : room per orderId, emit 'location-update' every 5 seconds with rider lat/lng. Firebase Cloud Messaging integration for push notifications: send on status change (preparing, picked_up, en_route, delivered)"	Tracking + push APIs
Frontend Web	Build order tracking page.	"Generate React OrderTracking component: step progress bar (5 steps), live map with rider marker using Mapbox GL, ETA countdown, contact rider button (masked phone)"	Tracking page
Mobile	Same + push notification handling.	"Generate React Native OrderTracking screen with react-native-maps, setBackgroundMessageHandler for FCM notifications when app is killed"	Tracking + push
UI/UX	Design tracking screen with step progress bar.	"Design tracking screen: top progress bar (Confirmed → Prepared → Picked → En route → Delivered), map view, rider info card, ETA timer"	Tracking designs

DevOps	Setup WebSocket secure (WSS) + auto-reconnect.	Configure Nginx with proxy_pass for WebSocket → Add proxy_set_header Upgrade \$http_upgrade → Configure Socket.io client with reconnection (max 5 attempts, exponential backoff)	WebSocket active
--------	--	--	------------------

Profile & History

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Order History Service + Reorder function.	"Generate GET /orders with pagination (limit 20, offset), filters (status, date range). POST /orders/:id/reorder (duplicate last order items to cart). Wallet API: GET /wallet/balance, POST /wallet/withdraw"	History + reorder APIs
Frontend Web	Build profile page.	"Generate React ProfilePage: tabs (Order History with filters, Saved Addresses with edit/delete, Payment Methods with add/remove, Language Selector (5 languages), Delete Account with confirmation modal, WAWU+ Subscription with pricing cards"	Profile page
Mobile	Same + share order status via SMS.	"React Native ProfileScreen + expo-sms to share order status: opens SMS with 'My order #123 is on its way! Track: wawu.africa/track/123'"	Profile + share
DevOps	Backup strategy: daily PostgreSQL dump to S3.	Create cron job: 0 2 * * * pg_dump wawu > backup.sql && aws s3 cp backup.sql s3://wawu-backups/\$(date +%Y-%m-%d).sql → Retention policy: 30 days	Automated backups

End of Phase 2: Complete user journey from browse → checkout → track → reorder. All 34 user functions working.

Phase 3: Vendor & Agent Portals

Vendor Order Management

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Vendor Dashboard API.	"Generate vendor APIs: GET /vendors/orders/live (orders with status pending/confirmed/preparing), PUT /vendors/orders/:id/accept, PUT /vendors/orders/:id/decline, PUT /vendors/orders/:id/ready, GET /vendors/dashboard/today (orders count, revenue, rating)"	Vendor APIs
Frontend Web	Build vendor dashboard.	"Generate React VendorDashboard: live orders section with SSE (Server-Sent Events) for real-time updates, each order card shows items, customer info, special instructions, buttons (Accept/Decline/Ready), print ticket button opens print dialog"	Vendor dashboard
Mobile	Vendor mobile lite (basic notifications).	"Generate React Native VendorLite: push notifications for new orders, tap to see order details, buttons to accept/decline (minimal)"	Vendor mobile lite
DevOps	Setup separate vendor database schema (RLS).	"Enable Row Level Security on vendors table: CREATE POLICY vendor_isolation ON vendors USING (user_id = current_user_id())"	Vendor isolation

Vendor Analytics

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Analytics Service.	"Generate analytics APIs: GET /vendors/analytics/sales (daily/weekly/monthly aggregates), GET /vendors/analytics/trend (order count trend 30 days), GET /vendors/analytics/heatmap (hourly distribution), GET /vendors/analytics/geography (customer delivery locations), GET /vendors/analytics/cancellations (rate + reasons)"	Analytics APIs
Frontend Web	Build vendor analytics dashboard.	"Generate React AnalyticsDashboard: Chart.js line charts for sales trend, bar charts for peak hours, heatmap for geography (Mapbox), donut chart for cancellation reasons, date range picker, CSV export button"	Analytics dashboard
Mobile	Not needed (web only)	—	—

DevOps	Setup read replica for analytics queries.	"Create RDS read replica → Configure Prisma to use replica for analytics queries: datasource db { url = env("DATABASE_URL_REPLICA") }"	Read replica
--------	---	--	--------------

Vendor Inventory Management

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Inventory Service.	"Generate inventory APIs: PUT /vendors/inventory/:productId (update stock), POST /vendors/alerts/low-stock (set threshold, email/push on crossing), batch tracking for produce/livestock (harvest_date, slaughter_date), bulk update via CSV"	Inventory APIs
Frontend Web	Build inventory management.	"Generate React InventoryPage: product table with stock levels, low stock badge, edit inline, bulk upload CSV button, low stock alert settings modal"	Inventory pages
Mobile	Not needed.	—	—
DevOps	Audit log for all inventory changes.	"Add trigger: CREATE TRIGGER inventory_audit AFTER UPDATE ON inventory FOR EACH ROW EXECUTE FUNCTION log_inventory_changes()"	Inventory audit

Vendor Payouts

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Payout Service.	"Generate payout APIs: GET /vendors/payouts (history with status), POST /vendors/payouts/request (amount, preferred method), GET /vendors/commissions (breakdown per order), reconcile commission daily via cron"	Payout APIs
Frontend Web	Build payout section.	"Generate React PayoutPage: earnings summary chart (YTD), request payout button, history table (date, amount, status, invoice link), invoice PDF download (pdf-lib)"	Payout pages
Mobile	Not needed.	—	—
DevOps	Idempotency keys for payouts (prevent double payout).	"Add idempotency_key column to payouts table with UNIQUE constraint → Client generates UUID and retries use same key → Duplicate returns existing payout"	Payout safety

Agent Service Foundation

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Agent Service.	"Generate agent APIs: POST /agents/register (device fingerprinting, admin approval), GET /agents/pending (admin approval queue), PUT /agents/:id/approve (admin), GET /agents/commission (per transaction breakdown), commission payout calculation (0.5% of transaction volume)"	Agent APIs
Frontend Web	Build agent management table.	"Generate React AgentManagement: pending agents table (name, phone, location, request date), approve/decline buttons, active agents list with performance metrics (transactions, commission earned)"	Agent admin
Mobile	Agent app foundation.	"Generate React Native AgentApp: login screen, bottom tabs (Dashboard, Traders, Transactions, Earnings), Dashboard shows today's transactions count and commission, offline warning banner"	Agent app base
DevOps	Device fingerprinting (prevent duplicate agent accounts).	"Generate fingerprint from: userAgent, platform, language, screen resolution, timezone. Hash with SHA256. Store unique constraint."	Fingerprinting

Agent Offline Sync & Cash Payout

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Offline Sync Service.	"Generate sync APIs: POST /agents/sync (batch upload offline transactions, conflict resolution rules: latest timestamp wins, admin alerted on conflict), GET /agents/pending-sync-count, POST /agents/cash-payout (record cash given to trader, trigger reimbursement to agent via mobile money API)"	Sync APIs
Frontend Web	Not needed.	—	—
Mobile	Complete agent offline flow.	"Generate React Native OfflineAgent: SQLite schema (traders, transactions, photos, signatures), sync queue with background sync (checks network every 5 minutes), camera capture with timestamp overlay, signature capture using react-native-signature-canvas, push notification listener for admin alerts"	Full agent app

UI/UX	Design offline sync status indicator.	"Design offline indicator with sync button, pending count badge, loading skeleton for sync, success/failure toast messages"	Offline designs
DevOps	Encrypt offline SQLite database.	"Use react-native-sqlite-storage with encryption: <code>sqlite.openDatabase({ name: 'wawu.db', key: deviceFingerprint })</code> "	Encrypted offline

End of Phase 3: Vendor dashboard fully functional. Agent app can onboard traders offline. All 52 vendor+agent functions working.

Cross-Border Trade & Escrow

Farmer Export Listings

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Export Service.	"Generate export APIs: POST /exports/listings (farmer posts inventory: product, quantity, price, harvest date, location), GET /exports/prices (aggregate prices across corridors from transaction history), GET /exports/listings (filter by product, corridor, date)"	Export APIs
Frontend Web	Build farmer listing page.	"Generate React FarmerPortal: post inventory form (product autocomplete, quantity, price, harvest date picker, location map picker), active listings table (edit/delete/mark sold), market prices table (product, Nigeria price, Benin price, Togo price, Niger price, Cameroon price)"	Farmer pages
Mobile	Basic view for farmers (minimal).	"Generate React Native FarmerBasic: view own listings, receive push notifications when buyer inquires"	Limited mobile
DevOps	SQL injection prevention (already parameterized via Prisma).	—	Verified

Escrow & multi-Currency

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Escrow Service + Mobile Money Integration.	"Generate escrow APIs: POST /escrow/hold (buyer deposits CFA via Partner API or Naira via card, create escrow record), POST /escrow/release (buyer confirms delivery, release to seller), POST /escrow/dispute (open dispute with evidence), scheduled job (release after 7 days if no action). Integrate MTN MoMo API for CFA deposits."	Escrow APIs
Frontend Web	Build buyer escrow flow.	"Generate React EscrowFlow: deposit screen (amount, currency selector CFA/NGN, payment method), confirmation screen with escrow ID, status tracker (Held → Released/Disputed)"	Escrow UI
Mobile	Same as web.	—	Escrow mobile
DevOps	Escrow funds in separate account.	"Create separate bank account for escrow → Daily reconciliation job: SELECT SUM(amount_held) FROM escrow WHERE status='held' → Compare with bank balance → Alert if mismatch > ₦10,000"	Escrow safety

Currency Conversion

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Currency Conversion Service.	"Generate FX API: GET /fx/rate (from=NGN&to=XOF) → fetch from multiple providers (ExchangeRate-API, OpenExchangeRates), fallback if one fails, cache with 5-minute TTL, apply WAWU spread 2.5%, POST /fx/convert (amount, from, to) → return converted amount with fee breakdown"	FX API
Frontend Web	Show conversion preview.	"Generate React ConversionPreview: input amount, select currencies, real-time conversion display, WAWU fee breakdown, 'You save X% vs black market' tooltip"	Conversion preview
Mobile	Same.	—	Conversion mobile
DevOps	Rate caching to avoid API overuse.	"Implement Redis cache with 5-minute TTL for /fx/rate → Reduces API calls from 10,000/day to 288/day"	Cached rates

Transport Dispatch

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Transport Dispatch Service.	"Generate transport APIs: POST /transport/loads (trader posts load: origin, destination, weight, offered price), GET /transport/loads/available (for drivers), POST /transport/loads/:id/bid (driver offers price/ETA), PUT /transport/loads/:id/accept (trader accepts bid), POST /transport/trips/:id/start, POST /transport/trips/:id/checkpoint, POST /transport/trips/:id/complete, GET /transport/earnings"	Transport APIs
Frontend Web	Build transport request page.	"Generate React TransportPage: post load form (origin/destination autocomplete, weight slider, offered price), view active bids table (driver name, price, ETA, accept button), trip status tracker with checkpoint timeline"	Transport web
Mobile	Driver app.	"Generate React Native DriverApp: load board (available loads list with filters), bid modal (price, ETA, notes), active trip screen with checkpoint buttons (GPS required), earnings screen (per trip, daily, weekly totals), contact trader button (masked phone via Twilio Proxy)"	Driver app
DevOps	GPS spoofing detection.	"Calculate speed between checkpoints using Haversine formula: distance / time elapsed. If speed >120km/h or physically impossible (e.g., 100km in 10 minutes), flag for review."	Anti-spoofing

Credit Scoring

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Credit Scoring Service.	"Generate credit score calculation: factors (transaction volume 40%, dispute rate 30%, consistency 20%, tenure 10%). Score range 0-1000. Weekly cron job recalculates for all traders. Store historical scores. GET /credit-score (current + history)"	Credit API
Frontend Web	Build credit score view.	"Generate React CreditScorePage: gauge visualization (0-1000), score breakdown (factors percentages), loan eligibility badge, improvement tips (e.g., 'Complete 5 more transactions to increase score')"	Credit page
Mobile	Basic view (score only).	"React Native CreditScoreCard: score number, gauge, short message ('Good'/'Fair'/'Poor')"	Credit mobile
DevOps	Encrypt credit data at rest.	"Enable column-level encryption for credit_score in PostgreSQL: ALTER TABLE users ALTER COLUMN credit_score SET DATA TYPE bytea USING pgp_sym_encrypt(credit_score::text, 'key')"	Encrypted

Microfinance Integration

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Microfinance API integration.	"Generate loan APIs: POST /loans/apply (user applies with amount, purpose, repayment period; WAWU sends trader profile + credit score to partner bank via webhook), POST /webhooks/bank/loan-status (bank updates loan status: approved/rejected/disbursed), POST /loans/repay (deduct from escrow automatically on confirmation)"	Bank APIs
Frontend Web	Build loan request flow.	"Generate React LoanPage: loan calculator (amount slider, repayment period, estimated monthly payment), application form (purpose of loan, business registration number), status tracker (Pending → Under Review → Approved → Disbursed)"	Loan pages
Mobile	Not needed.	—	—
UI/UX	Design loan application + approval timeline.	"Design loan application: step indicator (1. Amount, 2. Purpose, 3. Confirm), document upload area, status timeline with icons and dates"	Loan designs
DevOps	Webhook signature verification.	"Generate secret per bank → Bank signs webhook payload with HMAC-SHA256 using secret → WAWU verifies signature before processing"	Webhook security

End of Phase 4: Cross-border trade fully functional. Escrow holds/releases funds. Credit scores generated. All 37 cross-border functions working.

Admin & Data Monetization

Admin User Management

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Admin Audit Service.	"Generate admin APIs: PUT /admin/vendors/:id/approve, PUT /admin/agents/:id/approve, PUT /admin/riders/:id/approve, PUT /admin/users/:id/suspend, PUT /admin/vendors/:id/commission (override with effective date), POST /admin/zones (GeoJSON polygon), POST /admin/promos (code, discount type, max uses, expiry), PUT /admin/features (toggle maintenance mode, beta features). Audit log: automatically log all admin actions (who, what, when, IP, old value, new value)"	Admin APIs
Frontend Web	Build admin dashboard.	"Generate React AdminDashboard: tabs (Vendor Approvals, Agent Approvals, User Management, System Settings, Promos, Feature Flags. Each table has search, filter, pagination, approve/decline/suspend buttons. Audit log viewer with export CSV."	Admin dashboard
Mobile	Not needed.	—	—
DevOps	Admin MFA (mandatory TOTP).	"Integrate speakeasy for TOTP: POST /admin/mfa/setup (generate secret, QR code), POST /admin/mfa/verify (verify TOTP, enable MFA). All admin routes require MFA token in header."	Admin MFA

Dispute Resolution & Reconciliation

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Dispute Resolution Service + Reconciliation.	"Generate dispute APIs: GET /admin/disputes (list with filters: status, order_id, date), GET /admin/disputes/:id/evidence (photos, chat logs, transaction history), POST /admin/disputes/:id/resolve (action: release_escrow_to_seller, refund_buyer, adjust_balances). Reconciliation: GET /admin/reconciliation (compare escrow table sum vs bank balance, alert if mismatch). Manual escrow adjustment: POST /admin/escrow/adjust (admin only, with reason, audit logged)."	Dispute + reconciliation APIs
Frontend Web	Build dispute dashboard.	"Generate React DisputeDashboard: ticket list with priority badges, ticket detail view with evidence gallery (photos, transaction timeline), resolution modal (select action, add notes). Reconciliation dashboard: metrics cards (Total Held, Bank Balance, Difference), history table of adjustments."	Dispute + reconciliation

Mobile	User open dispute via order history.	"React Native OrderDetail: Report Issue button, form (reason dropdown, photo upload, description), submit to dispute API"	Dispute mobile
UI/UX	Design dispute thread with photo gallery.	"Design dispute thread: timeline view (User: 'Goods not delivered' → Rider: 'I delivered to address' → Admin: 'Reviewing evidence'), photo gallery with zoom, resolution badge."	Dispute designs
DevOps	Immutable dispute decisions (no edit after 24h).	"Add constraint: resolution_notes cannot be updated after 24 hours of being set. CHECK (resolved_at > NOW() - INTERVAL '24 hours' OR resolution_notes IS NULL)"	Immutable disputes

Data Monetization API

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Data Monetization API.	"Generate data APIs for government: GET /data/trade/aggregate (filters: product_id, origin_corridor, destination_corridor, start_date, end_date; returns aggregated volume, value, transaction count; trader identity hashed, no PII), GET /data/border/live (real-time dashboard: today's volume by border, top products, price trends), GET /data/reports/monthly (PDF report with charts), POST /data/str/sync (push anonymized transaction to government STR system via authenticated API). Multi-user government accounts with RBAC (NEPC, Customs, NAFDAC). API key management with expiry and IP whitelist."	Data APIs
Frontend Web	Build government subscriber portal.	"Generate React GovernmentPortal: dashboard (filter by corridor, date range, product), charts (trade volume trend, top products), map (borders with heatmap), export buttons (PDF, CSV), API key management page (create/revoke keys, whitelist IPs)"	Government portal
Mobile	Not needed.	—	—
UI/UX	Design government dashboard with map.	"Design government dashboard: white-label for each agency, large metric cards (Today's Trade Volume, Active Traders, Top Corridor), interactive map (border markets with popups), date range selector"	Government designs
DevOps	API key security + IP whitelisting.	"API keys stored as bcrypt hash. Middleware: validate key, check expiry, check request IP against whitelist (if configured). Rate limit: 1000 requests/minute per key."	API key security

Policy Tool & Webhooks

Role	Tasks	Code Generation Prompt	Deliverable
Backend	Generate Policy Simulation Tool + Webhook Service.	"Generate policy simulation: POST /data/policy/simulate (parameters: checkpoint_count, fuel_price, road_closure_days; model uses historical trip time data to project trade volume change, returns percentage impact). Webhook service: POST /webhooks (register webhook URL, events, secret), POST /webhooks/:id/trigger (test), automatic retry on failure (exponential backoff, max 5 attempts), idempotency keys (prevent duplicate processing)."	Policy API + webhook service
Frontend Web	Build policy tool interface + webhook console.	"Generate React PolicyTool: sliders for variables (checkpoint count, fuel price), live projected volume change chart, 'Generate Report' button. Webhook Console: registered webhooks table, test button, delivery logs (status, response, retries)."	Policy UI + webhook console
Mobile	Not needed.	—	—
UI/UX	Design policy simulation slider.	"Design policy tool: left panel with sliders (real-time updates), right panel with projection chart (before/after comparison), export PDF button"	Policy designs
DevOps	Idempotency key implementation.	"Webhook processor checks idempotency_key in request header → If key exists in Redis, return previous response → Else process and store response with key (TTL 24 hours)"	Idempotency

End of Phase 5: Admin controls everything. Government can pull data via API. All 31 admin+government functions working.

Security Hardening & Testing

Security Scan & Hardening

Role	Tasks	Commands / Configuration	Deliverable
Backend	Run OWASP ZAP scan. Fix findings. Add Helmet.js, CORS strict.	<code>docker run -t owasp/zap2docker-stable zap-full-scan.py -t https://staging.wawu.africa -r zap_report.html</code> → Add <code>helmet()</code> middleware → Configure CORS: <code>cors({ origin: ['https://wawu.africa', 'https://staging.wawu.africa'] })</code>	Hardened backend
Frontend Web	Implement CSP headers. Sanitize inputs.	Add CSP header: Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline' https://maps.googleapis.com → Use DOMPurify for all user-generated content	Hardened web
Mobile	SSL pinning. Secure storage.	iOS: Add keys to Info.plist NSAppTransportSecurity → Android: <code>network_security_config.xml</code> with pins → Use <code>react-native-encrypted-storage</code> instead of <code>AsyncStorage</code>	Hardened mobile
UI/UX	No changes.	—	—
DevOps	Setup WAF (AWS WAF). Enable DDoS protection.	AWS WAF rules: SQL injection, XSS, size constraint (5MB), rate-based (1000 per 5 minutes) → AWS Shield Advanced for DDoS	WAF active

Load Testing & Optimization

Role	Tasks	Commands / Configuration	Deliverable
Backend	Load testing with K6. Optimize slow queries.	<code>k6 run -u 1000 -d 5m load-test.js</code> (test script with user journey: login → search → add to cart → checkout) → EXPLAIN ANALYZE on slow queries → Add missing indexes	Load test report (<1% error, p95 <500ms)
Frontend Web	Bundle optimization. Lighthouse >90.	<code>npm run build -- --profile</code> → Enable code splitting: <code>lazy(() => import('./HeavyComponent'))</code> → Tree shaking confirmed → Run Lighthouse CI	Lighthouse score >90

Mobile	App size <50MB. Offline mode tested.	npx expo optimize → Enable Hermes engine → Test offline mode (airplane mode: register trader, enter transaction, verify local storage)	App size <50MB
UI/UX	Accessibility audit (WCAG 2.1 AA).	Install axe DevTools → Run on all pages → Fix: color contrast, ARIA labels, keyboard navigation, focus indicators	Accessibility report (0 violations)
DevOps	Setup Prometheus + Grafana monitoring.	helm install prometheus prometheus-community/kube-prometheus-stack → Configure alerts: error rate >1% for 5min, p95 latency >1s for 10min → Slack webhook for alerts	Monitoring active

End of Phase 6: Security report with no critical findings. Performance SLA defined.